Laboratório de Redes de Computadores

PUCRS | ESCOLA POLITÉCNICA

Trabalho 1 -- Servidor HTTP/1.1 sobre sockets TCP

Objetivos

Este trabalho tem como objetivo o projeto, implementação e análise de um **servidor HTTP/1.1** construído diretamente sobre sockets TCP, atendendo clientes em **máquinas distintas** da rede.

O protocolo HTTP é estudado a partir da camada que o sustenta, o TCP, e a pergunta central que o trabalho deve responder é: **como o comportamento do TCP determina o desempenho percebido do HTTP?** As conexões persistentes são uma resposta direta ao custo de abrir uma conexão TCP. Esse custo só se torna visível quando existe um RTT real entre cliente e servidor, e é por isso que o trabalho **não** é executado em `localhost`.

O trabalho é dividido em duas partes progressivas. * A Parte 1 implementa o servidor. * A Parte 2 adiciona persistência de conexão, com as medições comparativas.

A entrega final consiste em um **relatório técnico** e no **código-fonte** do servidor. Adicionalmente, cada grupo deverá apresentar sua solução (somente para o professor) e participará de um **teste de interoperabilidade** com outro grupo durante a aula de apresentação.

Ambiente de Execução

O servidor executa em uma máquina do grupo e é acessado a partir de **outras máquinas** da rede.

- **Porta:** utilizar uma porta alta (acima de 1024).
- **Bind:** o servidor deve escutar em `0.0.0.0` (todas as interfaces), não em `127.0.0.1` (loopback).
- **Captura:** Wireshark na interface de rede da máquina, com filtro `tcp.port == <porta do servidor>`.
- **Cliente:** navegador e `curl`.

Verificação inicial (fazer antes de implementar)

Antes de escrever qualquer código, confirme que as máquinas do grupo se enxergam:

1. Descubra o endereço IP de cada máquina (`ipconfig`).
2. A partir de uma máquina, teste o alcance da outra (`ping <ip>`).
3. Confirme que o Wireshark lista e captura na interface de rede.

   Se as máquinas não se alcançarem, comunique ao professor **imediatamente**. As medições da Parte 2 dependem de comunicação entre máquinas distintas.

Restrições Técnicas

- **Linguagem:** livre, desde que execute na VDI sem privilégios de administrador e sem instalação de runtime adicional.

- **Sockets:** a comunicação deve usar a API de sockets TCP diretamente (`socket`, `bind`, `listen`, `accept`, `recv`, `send`).
- **Proibido:** qualquer biblioteca ou módulo que implemente HTTP do lado servidor (`http.server`, `Flask`, `Express`, `HttpListener`, etc.). O parsing e a geração das mensagens HTTP devem ser feitos pelo grupo.
- **Permitido:** bibliotecas da linguagem para manipulação de arquivos, datas, hashes e concorrência (threads).

Parte 1: servidor

Regras de implementação

O servidor recebe como argumentos de linha de comando, no mínimo, a **porta** e o **diretório raiz** a ser servido:

```
<executável> --port 8080 --root ./www
```

Parsing da requisição

- Receber a requisição do socket e interpretar a **linha de requisição** (`método`, `request-target`, `versão`) e as **linhas de cabeçalho**.
- O terminador de linha do HTTP é `CRLF` (`\r\n`); o fim da seção de cabeçalho é uma linha em branco.
- Decodificar *percent-encoding* no caminho (ex: `%20` → espaço).

  Atenção: O TCP entrega um *fluxo de bytes*. Uma chamada de `recv()` pode retornar uma requisição incompleta, uma requisição inteira, ou uma requisição e o início da próxima. O parser deve acumular bytes em um buffer até encontrar a linha vazia que termina as linhas de cabeçalho, e preservar o que sobrar para a requisição seguinte.

Métodos obrigatórios

- **GET** — retorna as linhas de cabeçalho e corpo.
- **HEAD** — retorna exatamente as mesmas linhas de cabeçalho do GET correspondente, **sem corpo**. O `Content-Length` deve refletir o tamanho que o corpo teria.
- Qualquer outro método: **405 Method Not Allowed**, com linha de cabeçalho `Allow: GET, HEAD`.

Respostas obrigatórias

Toda resposta deve conter:

- Linha de resposta com a versão `HTTP/1.1` e o código adequado.
- `Content-Length` correto, inclusive nas respostas de erro e nas respostas a HEAD.
- `Content-Type` derivado da extensão do arquivo. No mínimo: `.html`, `.css`, `.js`, `.json`, `.txt`, `.png`, `.jpg`, `.pdf`. Extensão desconhecida deve configurar como `application/octet-stream`.
- `Date` no formato IMF-fixdate, em GMT, conforme a RFC 9110.
- `Server` com um identificador do grupo.

Códigos de status obrigatórios

Código	Situação
`200 OK`	Requisição bem-sucedida
`400 Bad Request`	Requisição malformada (request line inválida, header sem `:`)
`403 Forbidden`	Caminho fora do diretório raiz

Código	Situação
`404 Not Found`	Arquivo inexistente
`405 Method Not Allowed`	Método não suportado

Segurança

O servidor **não pode**, sob nenhuma circunstância, servir um arquivo fora do diretório raiz configurado.

- Requisições como `GET /../../Windows/System32/drivers/etc/hosts` devem retornar **403**.
- O relatório deve demonstrar, com pelo menos **três tentativas de travessia distintas** (sendo ao menos uma contendo percent-encoding), que o servidor as rejeita.

Concorrência

- O servidor deve atender **múltiplas conexões simultâneas**, de máquinas distintas. Uma requisição lenta não pode bloquear as demais.
- A estratégia é livre (ex., thread por conexão ou I/O não bloqueante). A escolha deve ser justificada no relatório.
- Teste obrigatório: com duas máquinas requisitando o servidor ao mesmo tempo, ambas devem ser atendidas.

Parte 2: Conexões persistentes

Regras adicionais

- Em HTTP/1.1 a conexão é **persistente por padrão**. O servidor deve manter o socket aberto após responder, salvo se o cliente enviar `Connection: close`.
- Quando o cliente enviar `Connection: close`, o servidor responde com a mesma linha de cabeçalho e encerra a conexão.
- Implementar um **timeout de conexão ociosa** (sugestão: 5 segundos), após o qual o servidor fecha a conexão.
- Uma conexão persistente deve suportar **múltiplas requisições em sequência**.

Medição

A medição é feita **entre máquinas distintas**, com captura Wireshark ativa.

Antes de medir, registrar o **RTT médio** entre as máquinas (`ping`) para referência para a análise.

Realizar **10 requisições sequenciais** ao mesmo recurso, em dois modos:

Cenário	Modo
C1	Uma conexão nova por requisição (`Connection: close`)
C2	Uma única conexão persistente para as 10 requisições

Para cada cenário, extrair da captura: número de handshakes TCP completos, número total de pacotes, bytes totais trafegados e tempo total.

Entrega

Relatório

O relatório deve ser sucinto, em poucas páginas, que inclua os seguintes pontos:

1. **Descrição da arquitetura:** estrutura do servidor, estratégia de concorrência adotada e sua justificativa.
2. **Tabela de conformidade:** para cada código de status obrigatório, a requisição `curl` que o produz e a resposta obtida.
3. **Demonstração de segurança:** as três tentativas de travessia de diretório, com a requisição enviada e a resposta do servidor.
4. **Captura de uma transação completa:** captura Wireshark de um GET bem-sucedido feito **de outra máquina**, identificando o handshake TCP, o pacote que carrega a requisição, os pacotes de resposta e o encerramento da conexão.
5. **Evidência de atendimento simultâneo:** captura ou log mostrando duas máquinas atendidas ao mesmo tempo.
6. **RTT medido** entre as máquinas utilizadas.
7. **Tabela comparativa C1 vs C2**, com as quatro métricas, e a economia percentual de pacotes e de bytes proporcionada pela conexão persistente.
8. **Análise do overhead de conexão:** a partir da captura, quantificar quantos bytes e quantos pacotes são gastos apenas em abrir e fechar conexões TCP no cenário C1.
9. **Análise em função do RTT:** relacionar a diferença de tempo entre C1 e C2 com o RTT medido. Quantos RTTs a mais o cenário C1 gasta, e de onde eles vêm? A conta deve identificar o custo do handshake TCP de cada conexão nova.
10. **Conclusão:** em que condição de rede a conexão persistente traria um ganho ainda maior do que o medido? A resposta deve se apoiar na relação entre o número de conexões e o RTT.

Teste de Interoperabilidade

Durante a aula de apresentação, cada grupo realizará um teste com outro grupo:

- O **navegador** de uma máquina do grupo A acessa o **servidor** do grupo B, e vice-versa.
- Deve ser requisitada uma página HTML com recursos referenciados (ex: uma imagem), de modo que o navegador emita múltiplas requisições.
- O teste é bem-sucedido se a página for renderizada corretamente no navegador, com todos os recursos carregados.

Apresentação

Cada grupo apresentará o código da solução diretamente ao professor. Todos os integrantes devem estar presentes e ser capazes de responder sobre qualquer parte do código. Um grupo que não consiga explicar o próprio código não será avaliado como autor dele.

Regras de Entrega

- **Modalidade:** grupos de até 3 componentes, indicados no Moodle.
- **Formato:** arquivo `.zip` ou `tar`, entregue até o início da aula de apresentação, contendo:
- **Código-fonte** do servidor (sem binários, arquivos temporários ou de build).
- `README` com instruções de compilação/execução e descrição dos argumentos de linha de comando.
- `www/` — o diretório de teste utilizado, incluindo a página do teste de interoperabilidade.
- `capturas/` com os arquivos `.pcapng` dos cenários obrigatórios (ex: `c1.pcapng`, `c2.pcapng`).
- **Relatório em PDF**, documento único cobrindo as duas partes.
- **Apresentação:** data disponível no cronograma da disciplina.

Atenção: apenas um integrante de cada grupo deve realizar a entrega. Não serão aceitos trabalhos de pessoas que não estiverem em um grupo. Trabalhos que não compilam ou que não executam não serão avaliados. Todos os trabalhos serão analisados e comparados; em caso de cópia, todos os trabalhos envolvidos receberão nota zero.

SOBRE O USO DE IA: este trabalho é avaliado majoritariamente na apresentação e no teste de interoperabilidade, ambos presenciais. O uso de IA é permitido, mas código que o grupo não consiga explicar não será considerado de autoria do grupo, e o trabalho receberá nota **zero**, independentemente de funcionar.

Avaliação

Item	Peso
Implementação	30%
Parsing da requisição, incluindo tratamento do fluxo de bytes	8%
Códigos de status e conformidade das respostas	7%
Proteção contra travessia de diretório	5%
Concorrência funcional	5%
Conexões persistentes com timeout	5%
Relatório	25%
Descrição da arquitetura e demonstração de conformidade (Parte 1)	8%
Tabela comparativa C1 vs C2 com evidência em capturas	7%
Análise do overhead de conexão em função do RTT	10%
Teste de interoperabilidade	20%
Servidor do grupo atende corretamente o navegador de outro grupo	20%
Apresentação	25%

Referências

- RFC 9110 — HTTP Semantics
- RFC 9112 — HTTP/1.1
- Kurose & Ross, *Redes de Computadores e a Internet: Uma abordagem top-down* — Seção 2.2 (HTTP)